Analyzing and Implementing Divide-and-Conquer Algorithms: Merge Sort and Quicksort

Jatin Jaiswal

Student ID: [STUDENT ID]

Department of Computer Science, University of the Cumberlands

MSCS 532: Algorithms and Data Structures

Assignment 2

Professor [INSTRUCTOR NAME]

November 1, 2026

Analyzing and Implementing Divide-and-Conquer Algorithms: Merge Sort and Quicksort

Divide-and-conquer algorithms split a problem into smaller instances of itself, solve those recursively, and combine the results (Cormen et al., 2022). This paper analyzes merge sort and quicksort, solves their recurrences, and compares Python implementations of both on sorted, reverse-sorted, and random inputs. Code, raw measurements, and figures are in the GitHub repository at https://github.com/jaiswal29/MSCS532_Assignment2.

Merge Sort

Problem and Key Steps

Merge sort sorts an array $A[p..r]$ of n comparable keys. *Divide:* compute the midpoint $q = \lfloor (p + r)/2 \rfloor$. *Conquer:* recursively sort $A[p..q]$ and $A[q+1..r]$. *Combine:* MERGE the two sorted halves in a single left-to-right pass using two temporary arrays. Because MERGE takes the left element on ties, the sort is stable, but it is not in place: the temporaries need $\Theta(n)$ extra storage. The inner loop of the implementation (`merge_sort.py`) is shown below.

```
# As long as each of the arrays L and R contains an unmerged element,
# copy the smallest unmerged element back into A[p..r].
while i < n_L and j < n_R:
    if counter is not None:
        counter.comparisons += 1
    if L[i] <= R[j]:
        A[k] = L[i]
        i += 1
    else:
        A[k] = R[j]
        j += 1
    k += 1

# One of L or R is exhausted; copy the remainder of the other
# (no further comparisons are needed).
while i < n_L:
    A[k] = L[i]
    i += 1
    k += 1
while j < n_R:
    A[k] = R[j]
    j += 1
    k += 1
```

Recurrence and Its Solution

Dividing costs $\Theta(1)$, each half has size $n/2$, and MERGE costs $\Theta(n)$, so $T(n) = 2T(n/2) + \Theta(n)$, $T(1) = \Theta(1)$. *Master method:* $a = b = 2$, so $n^{\log_b a} = n = \Theta(f(n))$; Case 2 gives $T(n) = \Theta(n \lg n)$.

Recursion tree: level i has 2^i nodes of size $n/2^i$, so every level costs cn; with $\lg n + 1$ levels the total is $cn \lg n + cn = \Theta(n \lg n)$. *Substitution:* guess $T(n) \le dn \lg n$; then $T(n) \le 2d(n/2) \lg(n/2) + cn = dn \lg n - dn + cn \le dn \lg n$ for $d \ge c$ and all $n \ge 2$, with d also chosen large enough to cover the base cases $T(2)$ and $T(3)$; the lower bound $T(n) \ge d'n \lg n$ follows the same way.

Asymptotic Analysis

The recurrence is independent of input order because MERGE always copies all n elements, so best, average, and worst cases are all $\Theta(n \lg n)$: merge sort is both $O(n \lg n)$ and $\Omega(n \lg n)$ on every input. Only the comparison count varies, from about $(n/2) \lg n$ on already-ordered input to at most about $n \lg n$ (exactly $n \lg n - n + 1$ when n is a power of 2). Since every comparison sort needs $\Omega(n \lg n)$ comparisons in the worst case (Cormen et al., 2022), merge sort is asymptotically optimal. Its predictability and stability make it the basis of external sorting and of Timsort, the run-exploiting merge sort behind Python's `sorted()` (Peters, 2002); its cost is $\Theta(n)$ extra memory and the copying that implies.

Quicksort

Problem and Key Steps

Quicksort (Hoare, 1962) does its work in the divide step. *Divide:* PARTITION rearranges $A[p..r]$ around a pivot x so that $A[p..q-1] \le x = A[q] < A[q+1..r]$. *Conquer:* recursively sort the two sides. *Combine:* nothing, since the sides are already in their final positions. The textbook PARTITION (`quick_sort.py`, below) always picks $x = A[r]$; RANDOMIZED-PARTITION first swaps a uniformly random element into $A[r]$. Both sort in place.

```
def partition(A: List, p: int, r: int, counter: Optional[Counter] = None) -> int:
    """PARTITION(A, p, r) -- Lomuto partition around the pivot x = A[r].

    Maintains the loop invariant (CLRS Figure 7.2):
        A[p..i]     <= x   (the "low side")
        A[i+1..j-1] >  x   (the "high side")
        A[r]        =  x   (the pivot)
    Each of the r - p iterations does exactly one key comparison, so
    PARTITION runs in Theta(n) time where n = r - p + 1.
    Returns the final index q of the pivot.
    """
    x = A[r]                    # the pivot
    i = p - 1                   # highest index into the low side
    for j in range(p, r):       # process each element other than the pivot
        if counter is not None:
```

```
            counter.comparisons += 1
        if A[j] <= x:           # does this element belong on the low side?
            i += 1              # index of a new slot in the low side
            A[i], A[j] = A[j], A[i]   # put this element there
    A[i + 1], A[r] = A[r], A[i + 1]   # pivot goes just right of the low side
    return i + 1                # new index of the pivot
```

Recurrences and Their Solutions

PARTITION costs $\Theta(n)$, so a split into sizes k and $n - k - 1$ gives $T(n) = T(k) + T(n - k - 1) + \Theta(n)$. *Worst case* (one side empty at every level, exactly what sorted and reverse-sorted input produce with the last-element pivot): $T(n) = T(n - 1) + \Theta(n)$. The master method does not apply, since the subproblem is not a constant fraction of n. The recursion tree is a path with costs cn, $c(n - 1)$, …, c, summing to $cn(n + 1)/2 = \Theta(n^2)$. Substitution with the guess $T(n) \le dn^2$ gives $d(n - 1)^2 + cn = dn^2 - (2d - c)n + d \le dn^2$ for $d \ge c$, $n \ge 1$. *Best case* (balanced split): $T(n) = 2T(n/2) + \Theta(n)$ is the merge-sort recurrence, so the master method (Case 2) and the recursion tree both give $\Theta(n \lg n)$; any constant-proportion split, even 9-to-1, has depth $\log_{10/9} n$ and is still $\Theta(n \lg n)$. *Expected case* with a random pivot: each rank is equally likely, so $E[T(n)] = (2/n) \Sigma_{q=0}^{n-1} E[T(q)] + \Theta(n)$, which substitution solves as $O(n \lg n)$; counting comparisons directly, the expectation is $2n \ln n + O(n) \approx 1.39\, n \lg n$ (Sedgewick, 1978).

Asymptotic Analysis

Quicksort is therefore $O(n^2)$ and $\Omega(n \lg n)$, with no single Θ bound covering all inputs: the worst case is $\Theta(n^2)$, the best case $\Theta(n \lg n)$, and the randomized version runs in expected $\Theta(n \lg n)$ on *every* input, its $O(n^2)$ worst case having negligible probability. In practice it is usually the fastest comparison sort: in place, sequential in memory, with a one-comparison, one-swap inner loop (Sedgewick, 1978). Its costs are instability, a stack that can reach depth $\Theta(n)$, and a predictable pivot that adversarial input can exploit, which libraries counter with randomization, median-of-three pivots, or introsort's fallback to heapsort.

Implementation and Experimental Comparison

Method

Both algorithms were implemented in Python 3.13.18 from the textbook pseudocode (`merge_sort.py`, `quick_sort.py`) and verified by unit tests covering textbook examples,

edge cases, duplicates, stability, and 200 random checks against `sorted()`. The harness `benchmark.py` ran merge sort, deterministic quicksort, randomized quicksort (fixed seed, so runs are reproducible), and, for calibration, the built-in `sorted()` on sorted, reverse-sorted, and uniformly random integers of 1,000, 2,000, 5,000, 10,000, 20,000, 50,000, and 100,000 elements, recording the median wall-clock time of five (three for n ≥ 50,000) trials (`time.perf_counter`, garbage collector disabled), the peak memory allocated during the call (`tracemalloc`; Python Software Foundation, 2025), and the key comparisons from one instrumented run. Deterministic quicksort on sorted and reverse-sorted input was capped at n = 20,000 because of its quadratic time. Measurements were taken on Linux (x86_64), 2 cores.

Results

Table 1

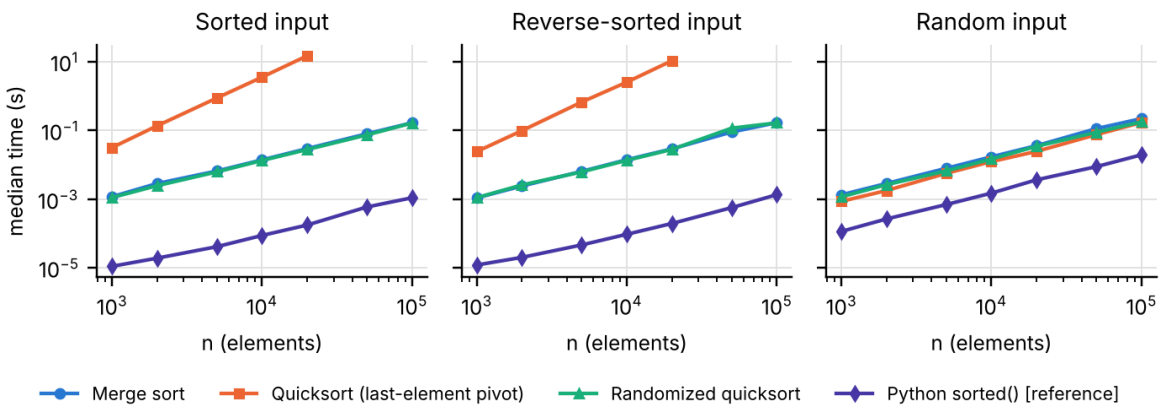
Median Running Time in Seconds by Algorithm, Input Order, and Input Size

Algorithm	Input	n = 10,000	n = 20,000	n = 100,000
Merge sort	Sorted	0.0138	0.0293	0.1655
	Reverse	0.0139	0.0287	0.1672
	Random	0.0169	0.0357	0.2187
Quicksort (last-element pivot)	Sorted	3.5228	14.7406	—
	Reverse	2.5108	10.5841	—
	Random	0.0123	0.0243	0.1683
Randomized quicksort	Sorted	0.0132	0.0275	0.1627
	Reverse	0.0133	0.0277	0.1642
	Random	0.0146	0.0343	0.1758
Python sorted()	Sorted	0.00009	0.00017	0.0011
	Reverse	0.00009	0.00019	0.0013
	Random	0.0015	0.0036	0.0193

Note. Dashes mark runs skipped because of $\Theta(n^2)$ running time. Python `sorted()` is Timsort in C, a reference point only.

Figure 1

Median Running Time Versus Input Size (Log–Log Axes)

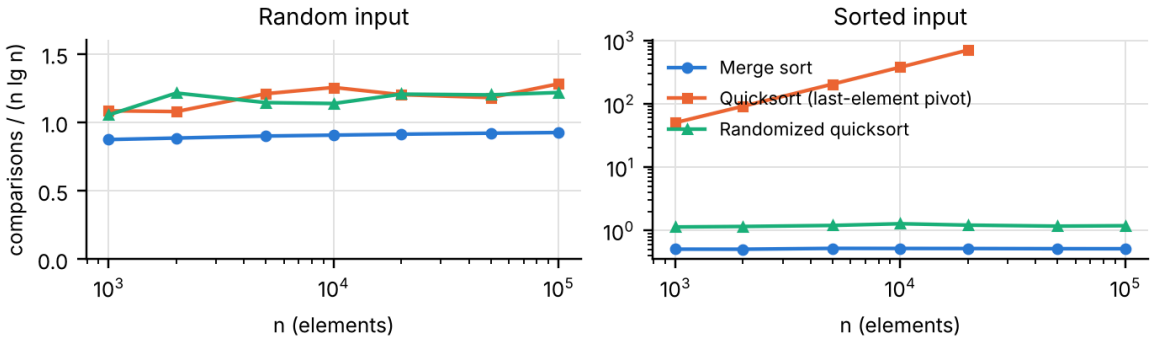


Note. On log–log axes a $\Theta(n \lg n)$ algorithm appears as a line of slope slightly above 1 and a $\Theta(n^2)$ algorithm as a line of slope 2.

Figure 1 and Table 1 confirm the analysis. Merge sort and randomized quicksort grow at the same rate on all three input orders: doubling n from 50,000 to 100,000 on random data multiplied their times by 1.96 and 2.01, close to the 2.13 of an exact $n \lg n$ law and far from the 4 of n^2 (three-trial medians at these sizes sit up to 15% above the fastest trial, so the second decimal is noise). Deterministic quicksort on sorted and reverse-sorted input doubled from 10,000 to 20,000 elements with factors of 4.18 and 4.22, the 4 predicted by $\Theta(n^2)$; at n = 20,000 it needed 14.7 s on sorted input versus 0.028 s for randomized quicksort and 0.029 s for merge sort, a 535-fold penalty for a change of pivot rule.

Figure 2

Key Comparisons Divided by n lg n for Random and Sorted Input



Note. A flat series indicates $\Theta(n \lg n)$ comparisons; the right panel uses a logarithmic vertical axis because the deterministic quicksort curve grows as $n/(2 \lg n)$.

Table 2

Key Comparisons at n = 20,000 and Their Ratio to n lg n

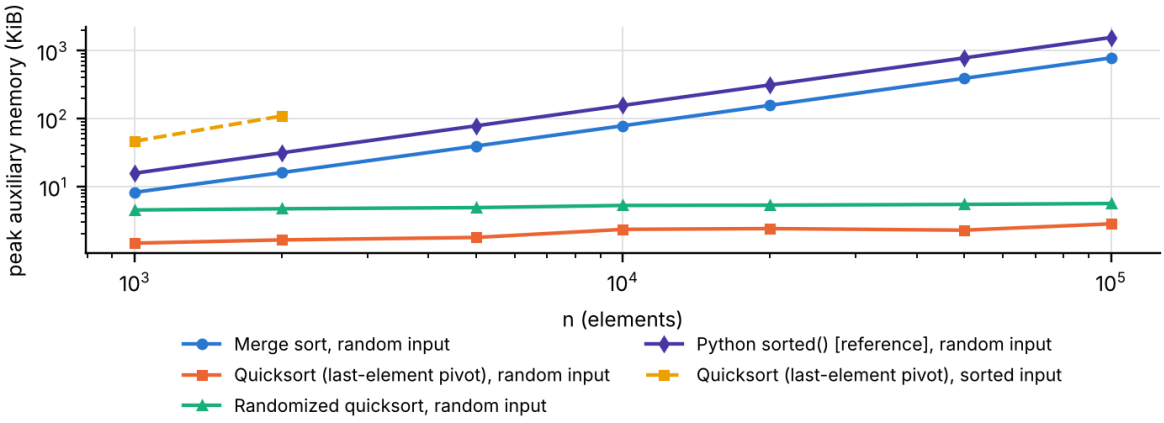
Algorithm	Input	Comparisons	Comparisons / *n* lg *n*
Merge sort	Sorted	148,016	0.52
	Reverse	139,216	0.49
	Random	260,866	0.91
Quicksort (last-element pivot)	Sorted	199,990,000	699.87
	Reverse	199,990,000	699.87
	Random	343,300	1.20
Randomized quicksort	Sorted	346,484	1.21
	Reverse	330,414	1.16
	Random	344,488	1.21

Note. *n* lg *n* = 285,754 at *n* = 20,000. The theoretical constants are 1 for the merge-sort worst case, 0.5 for its best case, and ≈1.39 for the expected case of randomized quicksort; the deterministic worst case is $n(n-1)/2$.

Table 2 and Figure 2 tie the measurements to the constants in the analysis: merge sort used 0.91 *n* lg *n* comparisons on random and 0.52 *n* lg *n* on sorted input, within the predicted 0.5 to 1; randomized quicksort used 1.21 *n* lg *n*, approaching 1.39 from below as the $-O(n)$ term predicts; and deterministic quicksort on sorted input performed exactly $n(n-1)/2 = 199{,}990{,}000$ comparisons.

Figure 3

Peak Auxiliary Memory Allocated During the Sort (Log–Log Axes)



Note. Measured with `tracemalloc`. The deterministic-quicksort series on sorted input stops at *n* = 2,000 because tracing every allocation of a $\Theta(n^2)$ run is prohibitively slow. Randomized quicksort's extra ≈2.5 KiB is the seeded generator's state; `sorted()` builds a second list (16 bytes per element).

Memory (Figure 3) also matches theory. Merge sort's peak auxiliary memory grew linearly to 782 KiB at n = 100,000, about 8 bytes per element, exactly MERGE's two temporary arrays of pointers. Deterministic quicksort on random input needed only 2.8 KiB at that size, the $\Theta(\lg n)$ stack of an in-place sort, but on sorted input its stack grew with n, from 47 KiB at n = 1,000 to 109 KiB at 2,000, the $\Theta(n)$ depth of the degenerate recursion tree.

Discrepancies Between Theory and Practice

The analysis does not predict several observations. First, on random input deterministic quicksort was the fastest Python implementation (0.168 s at n = 100,000 versus 0.219 s for merge sort) despite about 1.4 times *more* comparisons: asymptotic notation hides the constant factors of MERGE, which allocates two lists and copies every element at each of the lg n levels, while PARTITION swaps in place. Second, randomized quicksort was 4–48% slower than the deterministic version on random data across the seven sizes despite nearly identical comparison counts, because each of its $\Theta(n)$ partition calls pays for a Python-level `randint`. Likewise, sorted input cost deterministic quicksort 14.7 s against 10.6 s for reverse-sorted input at n = 20,000 with identical comparison counts, because on sorted input every comparison also triggers a (self-)swap. Third, merge sort was faster on sorted than on random input (0.165 versus 0.219 s) although both are $\Theta(n \lg n)$: it makes 56% as many comparisons, all perfectly predictable. Fourth, the built-in `sorted()` was 11 times faster than merge sort on random input because it is compiled C, and far faster on sorted input because Timsort merges existing runs in $\Theta(n)$ time. Finally, the analysis says nothing about an outright failure: under Python's default recursion limit of 1,000 frames, deterministic quicksort cannot sort even 1,000 already-sorted integers (output below), whereas randomized quicksort handles them easily; $\Theta(n)$ stack depth is a correctness problem, not just a cost.

```
$ python recursion_demo.py
default recursion limit: 1000
quick_sort(sorted 1000)             : RecursionError -- maximum recursion depth exceeded
randomized_quick_sort(sorted 1000)  : ok
after sys.setrecursionlimit(5000)   : quick_sort(sorted 1000) ok
```

Conclusion

The three recurrence methods agree: merge sort is $\Theta(n \lg n)$ on every input, and quicksort ranges from $\Theta(n \lg n)$ with balanced partitions to $\Theta(n^2)$ without them, randomization making the good case the expected one. The experiments reproduced these growth rates and comparison constants while exposing what the analysis omits: constant factors, the cost of randomization in Python, the speed of compiled code, and a recursion limit that turns a bound into a crash.

References

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Hoare, C. A. R. (1962). Quicksort. *The Computer Journal, 5*(1), 10–16. https://doi.org/10.1093/comjnl/5.1.10

Peters, T. (2002). *Timsort description* [Source file listsort.txt]. CPython repository. https://github.com/python/cpython/blob/main/Objects/listsort.txt

Python Software Foundation. (2025). *tracemalloc — Trace memory allocations*. Python 3.13 documentation. https://docs.python.org/3.13/library/tracemalloc.html

Sedgewick, R. (1978). Implementing Quicksort programs. *Communications of the ACM, 21*(10), 847–857. https://doi.org/10.1145/359619.359631